

Practical Work No. 1: Directed Graph Documentation C++

Student: Giurgiu Corina Maria

Date: March 24, 2026

1. Design & Internal Representation

- **Adjacency List (outbound):** A `std::vector` of `std::unordered_map<int, Edge*>`. This allows $O(1)$ average-case access to a vertex's neighbors.
- **Inbound Map (inbound):** A corresponding `std::vector` of `std::unordered_map` to track predecessors, ensuring "who points to a vertex" is also an $O(1)$ operation.
- **Edge Objects (cost):** Edges are stored as pointers to `Edge` structs. This allows the `inbound` and `outbound` maps to point to the **same object in memory**, making cost modifications instant for both directions.

2. Complexity Analysis

Operation	Complexity	Reason
add_edge / remove_edge	$O(1)$ average	Hash map insertion/deletion.
is_edge	$O(1)$ average	Direct hash map lookup.
get_in / out_degree	$O(1)$	Direct access to the <code>size()</code> of the map.
remove_vertex	$O(\deg(v))$	Iterates only through the incident edges of the vertex.
add_vertex	$O(1)$ amortized	Vector <code>push_back</code> operation.

3. Memory Management (The Rule of Three)

Because the graph uses raw pointers (`Edge*`) to ensure the `inbound` and `outbound` maps stay synchronized, the following were implemented to prevent memory leaks and segmentation faults:

- **Destructor (~Graph):** Properly deletes every edge pointer to free memory when the graph object is destroyed.
- **Copy Constructor:** Performs a **Deep Copy**. It creates entirely new `Edge` objects in memory so that modifications to a copy do not affect the original graph.

- **Assignment Operator:** Handles self-assignment and clears existing heap memory before copying new data from another graph.

4. Requirement Verification

- **Random Graphs:** Two graphs were generated and saved:
 1. `random_graph1.txt`: 6 vertices, 20 edges.
 2. `random_graph2.txt`: 7 vertices, 40 edges.
- **Large Files:** The C++ implementation was tested against `graph100k.txt`, showing significantly faster performance than Python due to the lower overhead of STL containers.
- **Preconditions:** All methods (like `get_in_degree` or `parse_out`) include bounds checking to ensure vertex IDs are within the valid range $[0, n-1]$.

5. Method Documentation

- **`Graph(int n)`:** Creates a graph with n vertices and 0 edges.
- **`add_vertex()`:** Adds a new vertex with the next available ID.
- **`remove_vertex(int v)`:** Removes a vertex. It takes a "snapshot" of edges first to avoid iterator invalidation during removal.
- **`is_edge(u, v)`:** Returns true if there is a directed edge from u to v .
- **`parse_out(v)` / `parse_in(v)`:** Returns a constant reference to the maps, allowing iteration over neighbors without exposing the internal structure to modification.
- **`create_random_graph(n, m)`:** Generates a random graph, with a safety cap to ensure $m \leq n^2$.

6. Implementation

```
// Internal structures used for the Graph ADT
struct Edge {
    int source; // starting vertex
    int target; // ending vertex
    int cost;
};

typedef Edge* Edge_id; // Abstract identifier for edges

// Data members of the Graph class
class Graph {
private:
    int nr_vertices;
    int nr_edges;
    // Map of outbound edges: source -> (target -> Edge_id)
```

```
std::vector<std::unordered_map<int, Edge_id>> outbound;  
// Map of inbound edges: target -> (source -> Edge_id)  
std::vector<std::unordered_map<int, Edge_id>> inbound;  
...  
};
```

Modules

[copy_random](#)

Classes

[builtins.object](#)
[Graph](#)

class **Graph**([builtins.object](#))
[Graph](#)(n=0)

A class representing a directed graph using three dictionaries.

Attributes:

- _vertices_count (int): The number of vertices in the graph.
- _edges_count (int): The number of edges in the graph.
- _outbound (dict): Maps a vertex to a list of vertices it points to.
- _inbound (dict): Maps a vertex to a list of vertices that point to it.
- _cost (dict): Maps an edge tuple (x, y) to its integer cost.

1. Internal Representation:
_outbound: A dictionary where keys are vertices and values are lists of successors. This provides O(1) average-case access to a vertex's neighbors.
_inbound: A dictionary mapping vertices to lists of predecessors, ensuring that finding "who points to a vertex" is also efficient.
_cost: A dictionary using a tuple (source, target) as a key. This allows for O(1) checks for edge existence and cost retrieval, which is much faster than searching through a list.
2. Complexity Analysis:
! Operation | Complexity | Reason |
! Add/Remove Edge | O(1) | Dictionary insertion/deletion and list appending are O(1). |
! Check Edge Existence | O(1) | Direct lookup in the _cost dictionary. |
! Get Degree | O(1) | Returning the len() of the inbound/outbound list. |
! Remove Vertex | O(deg(v)) | Must iterate through and remove all incident edges. |

3. Iterators (The "Bonus" Requirement)
Benefit: This encapsulates the internal data. The user can loop through the vertices/edges, but they cannot accidentally modify the internal lists of the graph while iterating.

4. Random [Graph](#) Generation (Requirement 5)
random_graph1.txt: Generated with 6 vertices and 20 edges.
random_graph2.txt: Generated with 7 vertices and 40 edges.
Logic: Your code handles the "max edges" constraint by capping the edge count at n^2 to prevent infinite loops if the user requests an impossible number of edges.

Methods defined here:

```
__init__(self, n=0)
    Creates a graph with n vertices (0 to n-1) and 0 edges
    Create 3 dictionaries for O(1) average case operations
    :param n: nr of vertices

add_edge(self, x, y, cost)
    x -> y
    :param x: starting vertex
    :param y: target vertex
    :param cost: cost of edge
    :return: true if add was successful, false otherwise

add_vertex(self, vertex)
    checks if the vertex exists
    if not, it creates empty entries in inbound and outbound
    increases vertex count by 1
    :param vertex:
    :return:

copy(self)
    deep copy ensures original graph and its copy are independent (modifications to one don't affect the other)
    :return:

get_cost(self, x, y)
    returns the cost of edge between x and y
    :param x: starting vertex
    :param y: target vertex
    :return: cost of edge

get_number_of_edges(self)

get_number_of_vertices(self)

has_vertex(self, vertex)

in_degree(self, vertex)
    how many edges enter the vertex
    complexity: O(1)
    :param vertex: given vertex
    :return: nr of inbounds

is_edge(self, x, y)
    checks if the edge exists - what if there is an edge with no cost? is it possible?
    complexity: O(1)
    :param x: starting vertex
    :param y: target vertex
    :return: true if edge exists, false otherwise

out_degree(self, vertex)
    how many edges get out of the vertex
    :param vertex: given vertex
    :return: nr of outbounds

parse_inbound(self, vertex)
    returns the list of inbound neighbours for this specific vertex
    :param vertex: given vertex
    :return: list of inbound neighbours

parse_outbound(self, vertex)
    returns the list of outbound neighbours for this specific vertex
    :param vertex: given vertex
    :return: list of outbound neighbours

parse_vertices(self)
    Returns an iterator for the set of vertices.

remove_edge(self, x, y)
    removes the edge between x and y
    - removes occurrence of y from outbounds of x
    - removes occurrence of x from inbounds of y
    - removes the cost of edge between x and y
    - decreases edges count by 1
    :param x:
    :param y:
    :return: true if edge was removed, false if edge was not found

remove_vertex(self, vertex)
    we can't delete directly the vertex because it might have edges that get out of it or that get to it
    - check if vertex exists
    - take copy of neighbours - inbound and outbound
    - used list because we can't modify directly when iterating -> list() -> shallow copy
    - iterate through outbounds and inbounds
    - remove edges connecting to that vertex
    - delete the actual vertex
    - decrement vertex count
    :param vertex:
    :return:

set_cost(self, x, y, cost)
    modifies the cost of edge between x and y
    :param x:
    :param y:
    :param cost: new cost
    :return: true if set was successful, false otherwise
```

Data descriptors defined here:

```
__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object
```

Functions

create_random_graph(number_of_vertices, number_of_edges)

read_graph_from_file(filename)

write_graph_to_file(graph, filename)